

Git mental model

This module is the picture you keep in your head while you use Git

Three places for your changes

- Git keeps three related views of a project
- The working tree is what you see and edit on disk
- The staging area, also called the index, is what you have prepared for the next snapshot
- HEAD is the last commit: the durable history you already recorded
- You move content forward with add, then commit
- You can move it back with restore when you need to undo a step safely

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Git mental model

INTERACTIVE TOOL

Git mental model

Watch bytes move between **working tree**, **index**, and **HEAD**. Edit → add → commit is one direction; restore undoes the other way.

Starter example: Clean tree — rtl/alu.v matches in all three layers. Edit the draft, then add and commit to push content toward HEAD.

Load starter example

Challenges 0 / 22 cleared

Quiz: three: Git's local model has how many "trees"? (number)

Answer

Show hintCheckNextIdle

Quiz: threeQuiz: indexQuiz: addQuiz: commitEdit diskStage itCommitQuiz: modifiedQuiz: stagedBoth diffsQuiz: restoreRestore workQuiz: unstageUnstageCommit ignores workQuiz: forwardNoop commitFull cycleQuiz: diskRe-addQuiz: why indexStarter clean

Three trees

HEAD c0ffee1 "alu skeleton" · 1 commit · file rtl/alu.v

clean — all three match

Working tree

files on disk · rtl/alu.v

module alu;
// add/sub
endmodule

git add

+

git restore

Index

staging area · next commit

module alu;
// add/sub
endmodule

git commit

+

restore --staged

HEAD

last commit snapshot

module alu;
// add/sub
endmodule

Git mental model lab starter

learn_git — git mental model

Real Git practice

```
wsl - module01-git-mental-model/examples/concepts

# Track A - real Unix
# real shell session (Track A)

$ git --version
git version 2.34.1

# practice repo: /tmp/git-mm-IQN6aG

$ git init
Initialized empty Git repository in /tmp/git-mm-IQN6aG/.git/

$ echo "notes" > note.txt

$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file> ..." to include in what will be committed)
    note.txt

nothing added to commit but untracked files present (use "git add" to track)

$ git add note.txt

$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file> ..." to unstage)
    new file:   note.txt

$ git commit -m "first snapshot"
[master (root-commit) 4e3b834] first snapshot
 1 file changed, 1 insertion(+)
 create mode 100644 note.txt

$ git log --oneline
4e3b834 first snapshot
```

Real shell — init, status, add, commit

Real Git practice — try these

```
# git --version — confirm Git is installed  
git --version
```

```
# git init — create a new repository here  
git init
```

```
# echo ... > note.txt — create a file in the working tree  
echo "notes" > note.txt
```

```
# git status — what is modified, staged, or untracked?  
git status
```

```
# git add note.txt — copy the change into the staging area  
git add note.txt
```

```
# git status — confirm it is staged for commit
```

Pitfalls to watch

- Do not treat the staging area as optional decoration, commit only records what you staged
- Do not confuse “saved in the editor” with “committed in Git.” And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, load the starter and finish a few challenges
- On real Git, repeat status, add, and commit in your practice tree
- When you are ready, take the short quiz, then continue to the next module